



RAG · RETRIEVAL QUALITY

Chunking strategies that improve RAG accuracy

Why most RAG failures start before retrieval even begins.



Ashwini

Senior AI/ML Engineer

Swipe →

What teams overlook

Most teams optimize:

✗ Embedding models

✗ Vector databases

✗ LLM selection

But retrieval quality depends heavily on one thing:

✓ **Chunking strategy**

pipeline

Documents → Chunks → Embeddings →
Vector DB → Retrieval → LLM

Bad chunks → Bad retrieval → Hallucinations



Why chunking exists

Large documents can't be embedded effectively as one giant vector.
Instead:

- 1 Split into chunks
- 2 Create embeddings
- 3 Store embeddings
- 4 Retrieve relevant chunks

The chunk is the atomic unit of retrieval.



Why naive chunking fails

Fixed character splitting causes:

✗ Broken sentences

✗ Lost headers

✗ Missing context

✗ Mixed topics

CHUNK 1

"Never run this migration in production because..."

CHUNK 2

"...it permanently deletes customer data."

The warning and instruction become **disconnected**.





Fixed size chunking

```
python

def fixed_size_chunk(text, size=500, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        end = start + size
        chunks.append(text[start:end])
        start = end - overlap
    return chunks
```

PROS

- ✓ Simple
- ✓ Fast
- ✓ Easy to build

CONS

- ✗ Breaks meaning
- ✗ Lower precision

BEST FOR Logs · transcripts · unstructured text



Sentence-based chunking

```
python

from nltk.tokenize import sent_tokenize

def sentence_chunk(text, max_t=500):
    chunks, cur = [], ""
    for s in sent_tokenize(text):
        if len(cur) + len(s) ≤ max_t:
            cur += " " + s
        else:
            chunks.append(cur.strip())
            cur = s
    return chunks
```

PROS

- ✓ Preserves meaning
- ✓ Better embeddings

CONS

- ✗ Uneven chunk sizes

BEST FOR Articles · blogs · reports



Semantic chunking

1. Embed every sentence
2. Compare neighbor similarity
3. Split when similarity drops

```
python

similarity = cosine_similarity(
    [prev_embedding],
    [current_embedding]
)[0][0]

if similarity < 0.75:
    create_new_chunk()
```

PROS

- ✓ Topic-aware splits
- ✓ Excellent retrieval

CONS

- ✗ Expensive preprocessing

BEST FOR Enterprise RAG · knowledge assistants



Hierarchical chunking

   structure

Document

|— Section

| |— Subsection

| |— Subsection

|— Section

   markdown

Authentication

OAuth Setup

Token Rotation

Each section becomes a chunk.

PROS

- ✓ Preserves structure
- ✓ Great for tech docs

CONS

- ✗ Needs clean headings

BEST FOR Documentation · papers · APIs



Ashwini

Senior AI/ML Engineer

Sliding window chunking

Chunk 1

Tokens 0–500

Chunk 2

Tokens 250–750

Chunk 3

Tokens 500–1000

Heavy overlap improves recall.

**Accuracy****Storage****Latency**

Best for: support systems · FAQ retrieval · conversational memory

**Ashwini**

Senior AI/ML Engineer

Production strategy

Modern production systems typically combine:

Document structure



Hierarchical chunking



Semantic splitting



20% overlap



Embedding + indexing

This hybrid usually outperforms any single strategy.



Ashwini

Senior AI/ML Engineer

Production example

```
python

chunks = chunk_by_headings(document)

semantic_chunks = semantic_split(
    chunks,
    similarity_threshold=0.75
)

final_chunks = apply_overlap(
    semantic_chunks,
    overlap_tokens=100
)
```

This is roughly how many **production RAG systems** chunk internal documentation.





Chunking isn't preprocessing. It's information architecture.

BETTER CHUNKING LEADS TO

✓ Higher Recall@K

✓ Lower hallucinations

✓ Lower token costs

✓ Better grounding

Garbage chunks → garbage retrieval → confident hallucinations.



Ashwini

Senior AI/ML Engineer

Follow